```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from imblearn.over_sampling import ADASYN
                        # 1. CREATE DEMO DATASET
def create_demo_data():
    np.random.seed(42)
    n = 1200
    data = pd.DataFrame({
        "age": np.random.randint(25, 80, n),
        "systolic_bp": np.random.randint(
            90, 190, n
        ),
        "diastolic_bp": np.random.randint(
            60, 120, n
        ),
        "body_fat": np.random.uniform(
            10, 45, n
        ),
        "smoking": np.random.randint(
            0, 2, n
        ),
        "physical_activity": np.random.randint(
            0, 2, n
        )
```

```
    })
    score = (
        (data["age"] > 55).astype(int)
        + (data["systolic_bp"] > 140).astype(int)
        + (data["diastolic_bp"] > 90).astype(int)
        + (data["body_fat"] > 30).astype(int)
        + data["smoking"]
        + (data["physical_activity"] == 0).astype(int)
    )
    data["risk"] = pd.cut(
        score,
        bins=[-1, 1, 2, 4, 10],
        labels=[0, 1, 2, 3]
    ).astype(int)
    return data
def train_model():
    data = create_demo_data()
    features = [
        "age",
        "systolic_bp",
        "diastolic_bp",
        "body_fat",
        "smoking",
        "physical_activity"
    ]
    X = data[features]
```

```
y = data["risk"]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
oversampler = ADASYN(
    random_state=42,
    n_neighbors=5
)
X_balanced, y_balanced = oversampler.fit_resample(
    X_train,
    y_train
)
X_balanced = pd.DataFrame(
    X_balanced,
    columns=features
)
                # ENSEMBLE MODEL 1 - RANDOM FOREST
rf = RandomForestClassifier(
    n_estimators=150,
    random_state=42
)
                # ENSEMBLE MODEL 2 - GRADIENT BOOSTING
```

```
    gb = GradientBoostingClassifier(
        random_state=42
    )
    rf.fit(
        X_balanced,
        y_balanced
    )
    gb.fit(
        X_balanced,
        y_balanced
    )
    return (
        rf,
        gb,
        features,
        X_balanced
    )
                    # 3. ENSEMBLE PREDICTION
def predict_risk(
    rf,
    gb,
    input_data
):
    rf_probability = rf.predict_proba(
        input_data
    )
```

```
gb_probability = gb.predict_proba(
    input_data
)
ensemble_probability = (
    rf_probability + gb_probability
) / 2
final_prediction = int(
    np.argmax(
        ensemble_probability,
        axis=1
    )[0]
)
risk_names = {
    0: "Low Risk",
    1: "Moderate Risk",
    2: "High Risk",
    3: "Very High Risk"
}
return (
    final_prediction,
    risk_names[final_prediction],
    ensemble_probability[0]
)
```